

PSI - 2.1 + 2.2

28.11.2025

Jakub Kwaśniak

Michał Pędziwiatr

Kacper Górski

Zadanie 2.1

Zrealizowano system komunikacji pomiędzy klientem napisanym w C (**tcp-client.c**) oraz serwerem TCP zaimplementowanym w Python (**tcp-server-2_1.py** i **tcp-server-2_2.py**).

Utworzona na potrzeby poprzednich zadań – 1.1 i 1.2 – struktura danych datagramu, została znacząco zmieniona (pliki **datagram.h** oraz **datagram.py**). Po modyfikacjach, przyjmuje formę węzła listy jednokierunkowej, które sekwencyjnie przesyłane są przez klienta w stronę serwera. Każdy z węzłów składa się z **16-bitowej liczby całkowitej**, **32-bitowej liczby całkowitej** oraz **napisu o zmiennej lecz ograniczonej długości**. Po stronie klienta, istnieje także **wskaźnik na następny węzeł** w liście, nie jest on jednak przesyłany przez sieć. Przed rozpoczęciem transmisji samych węzłów, klient przesyła także **4-bajtową liczbę całkowitą** w formacie big-endian, oznaczającą łączną liczbę węzłów w liście.

Serwer podobnie jak w poprzednim zadaniu działa na zasadzie nieskończonej pętli, nasłuchując na zadanym porcie. Po odebraniu danych, serwer odpakuje zawarte w niej liczby i napis, oraz wypisuje wartości te na wyjście konsoli

Zadanie 2.2 – wariant C

W ramach zadania zmodyfikowano implementację serwera, dodając obsługę współbieżności. Wprowadzono ją za pomocą wątków, używając modułu **threading**. Zmienia to logikę połączeń ze strony serwera, ponieważ po zaakceptowaniu nowego klienta (używając **accept()**),

główna pętla nie blokuje swojego dalszego działania. Zamiast tego tworzy osobny, nowy wątek dedykowany do obsługi nowego połączenia. Funkcja **handle_client()**, odpowiedzialna za odbiór nagłówka oraz węzłów od klienta, wykonywana jest równolegle. Dzięki temu mechanizmowi serwer ma możliwość komunikowania się z wieloma klientami jednocześnie, bez oczekiwania na zakończenie poprzedniego połączenia.

Napotkane problemy i ich rozwiązania

Podczas implementacji serwera TCP, napotkaliśmy na problem związany bezpośrednio z charakterystyką samego protokołu – strumieniowością danych. W przeciwieństwie do zadania 1.1 + 1.2, w ramach którego realizowaliśmy serwer UDP i korzystaliśmy z funkcji **recvfrom()**, w TCP pojedyncze wywołanie tej funkcji nie gwarantuje odebrania pełnego komunikatu.

Problem ten rozwiązaliśmy implementacją pomocniczej funkcji **_recv_all()**, która pobiera dane z gniazda w pętli dopóki nie uzbiera wymaganej liczby bajtów.

Testy

Testy odtworzyć można dzięki zapisie sesji terminala, przesłanym w pliku **demo_tcp_sleep.cast**. Prezentuje on działanie programu ze sztucznie wprowadzonym opóźnieniem (**time.sleep**), co umożliwia prezentację współbieżnej obsługi klientów po stronie serwera.

Poniższy fragment logów demonstruje wspomnianą współbieżność, poprzez naprzemienne przetwarzanie danych z adresu '**172.21.53.3**' (**klienta 1**) oraz **172.21.53.4** (**klienta 2**), przed zakończeniem któregośkolwiek z procesów:

```

z53_tcp_server | [SERVER] Listening on z53_tcp_server:2137
z53_tcp_server | [SERVER] Accepted ('172.21.53.3', 48742)
z53_tcp_server | [SERVER] Expecting 4 datagrams from ('172.21.53.3', 48742)
z53_tcp_server | [SERVER] Accepted ('172.21.53.4', 33770)
z53_tcp_server | [SERVER] Expecting 4 datagrams from ('172.21.53.4', 33770)
z53_tcp_server | [SERVER] Source: ('172.21.53.3', 48742), Decoded: 10, 100000, 'Node 1 (head)'
z53_tcp_server | [SERVER] Source: ('172.21.53.4', 33770), Decoded: 10, 100000, 'Node 1 (head)'
z53_tcp_server | [SERVER] Source: ('172.21.53.3', 48742), Decoded: 20, 200000, 'Node 2'
z53_tcp_server | [SERVER] Source: ('172.21.53.4', 33770), Decoded: 20, 200000, 'Node 2'
z53_tcp_server | [SERVER] Source: ('172.21.53.3', 48742), Decoded: 30, 300000, 'Node 3'
z53_tcp_server | [SERVER] Source: ('172.21.53.4', 33770), Decoded: 30, 300000, 'Node 3'
z53_tcp_server | [SERVER] Source: ('172.21.53.3', 48742), Decoded: 40, 400000, 'Node 4 (tail)'
z53_tcp_server | [SERVER] Source: ('172.21.53.4', 33770), Decoded: 40, 400000, 'Node 4 (tail)'

```

Widać także poprawne odpakowanie oraz wypisanie zawartości listy, serwer rozpoczyna transmisję od odczytania liczby węzłów, by następnie odczytać każdą z wartości (wersja bez sleep):

```

z53_client_1 | [CLIENT] Sending list of 3 elements...
z53_client_1 | [CLIENT] Sending message 1...
z53_client_1 | [CLIENT] Sending message 2...
z53_client_1 | [CLIENT] Sending message 3...
z53_client_1 | [CLIENT] Ended.
z53_tcp_server | [SERVER] Accepted ('172.21.53.3', 57398)
z53_tcp_server | [SERVER] Expecting 3 datagrams
z53_tcp_server | [SERVER] Decoded: 10, 100000, 'Node 1 (head)'
z53_tcp_server | [SERVER] Decoded: 20, 200000, 'Node 2'
z53_tcp_server | [SERVER] Decoded: 30, 300000, 'Node 3 (tail)'

```